

MESTRADO EM ROBÓTICA E SISTEMAS INTELIGENTES

Agente Autônomo para SI2 - Space Invaders

Documentação e Análise de Implementação

Guilherme Ribeiro - 114279
Francisco Fernandes - 115129

Unidade Curricular: Sistemas Inteligentes II

23 de junho de 2026

1 Introdução

Este documento descreve em detalhe a implementação de um agente autónomo para o ambiente **SI2 - Space Invaders**, uma plataforma de simulação em tempo real onde uma nave controlada pelo agente enfrenta ondas de “alienígenas” hostis que executam movimentos em forma de figura-8 e, ocasionalmente, mergulham em direção ao jogador.

O objetivo do agente é aprender a mover-se lateralmente (WEST/EAST) e a disparar estrategicamente para eliminar vagas de alienígenas, evitar colisões com alienígenas em mergulho, e maximizar a pontuação acumulada ao longo do tempo. A abordagem adotada foi o Q-Learning com exploração ϵ -greedy, utilizando uma representação de estado compacta e uma função de recompensa desenhada para promover o alinhamento com alvos e a eliminação de ameaças.

2 Visão Geral da Arquitetura

O projeto está organizado em torno de três camadas funcionais:

- **server/**: contém a lógica do jogo (`logic.py`), o servidor WebSocket (`server.py`) e os *assets* do *viewer* web (`viewer/`);
- **agents/**: contém a classe base (`base_agent.py`), o agente aleatório de referência (`dummy_agent.py`), o agente de controlo manual (`manual_agent.py`), o agente principal desenvolvido (`new_agent.py`) e o README com as instruções do projeto (`README.md`);
- **tests/**: contém testes unitários à lógica do jogo.

O fluxo de comunicação em cada iteração do ciclo principal segue a seguinte sequência:

1. O servidor envia ao agente um estado do tipo **state** ou **update** em formato JSON via WebSocket;
2. O agente recebe o estado em `BaseAgent.run` e invoca o método `deliberate`;
3. Em `deliberate`, o estado é abstraído numa tupla compacta (Secção 3);
4. A política ϵ -greedy seleciona uma ação a partir da Q-table (Secção 4);
5. A ação é enviada ao servidor como um dicionário JSON;
6. A recompensa é calculada e a Q-table é atualizada (Secção 5).

3 Ambiente e Representação de Estado

3.1 Descrição do Ambiente

O ambiente **SpaceInvaders** consiste numa grelha de 11×11 unidades. O jogador ocupa a posição $y = 0$ e move-se lateralmente entre $x = 0$ e $x = 10$. Os alienígenas, em número inicial de 10 (duas filas de cinco), executam trajetórias em figura-8 desincronizadas e, periodicamente, um deles entra em estado de mergulho (`is_diving=True`), rastreando ativamente a posição horizontal do jogador.

O estado observável pelo agente, tal como devolvido pelo servidor, inclui:

- `player_x`, `player_y`: posição atual do jogador;
- `lives`, `score`, `game_over`: estado do jogo;
- `lasers`: lista de lasers ativos;
- `aliens`: lista de alienígenas ativos, com posição, estado de mergulho e índice.

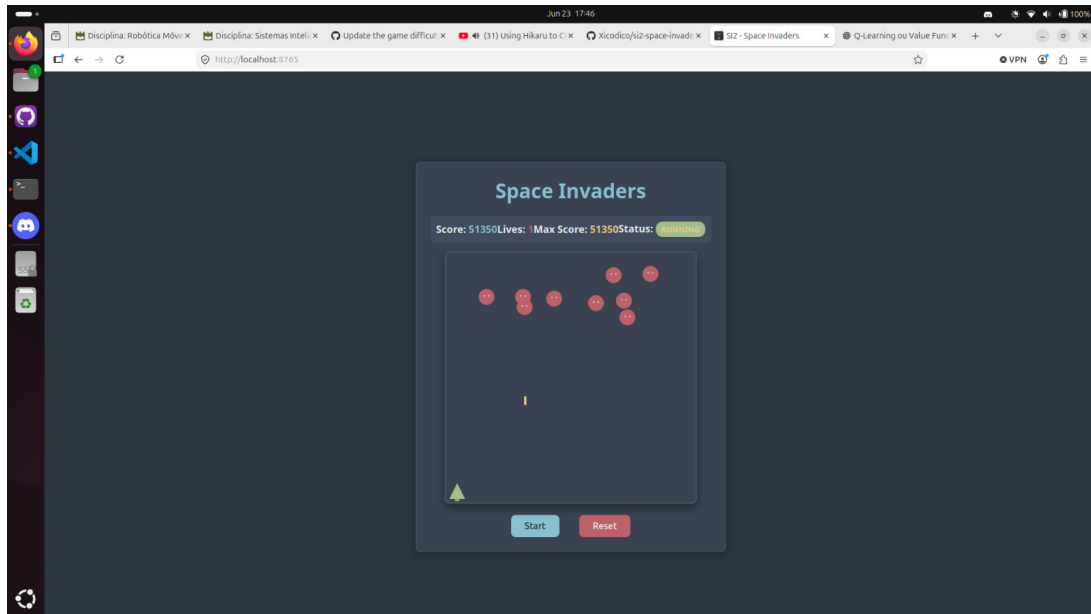


Figura 1: Exemplo de *gameplay* durante o modo de treino: a nave (triângulo verde) posiciona-se sob os alienígenas enquanto um laser está em voo.

3.2 Abstração do Estado

O espaço de estado bruto (posições em vírgula flutuante de 10 alienígenas, lasers, etc.) é demasiado grande para Q-Learning tabular. Por isso, o agente extrai seis variáveis para construir o estado:

1. `has_diving_alien` (booleano): indica se existe pelo menos um alienígena em mergulho ativo;
2. `aligned` (booleano): indica se o jogador está alinhado horizontalmente com o alienígena alvo ($|dx| \leq 1$);
3. `target_dx` (inteiro $\in [-10, 10]$): diferença horizontal entre o alienígena alvo e o jogador, limitada ao intervalo $[-10, 10]$;
4. `target_dy` (inteiro $\in [0, 10]$): metade da diferença vertical (arredondada) entre o alienígena alvo e o jogador, limitada ao intervalo $[0, 10]$;
5. `can_shoot` (booleano): indica se a ação de disparo está disponível, i.e., se o *cooldown* já expirou;

6. **danger** (booleano): indica se o agente está em perigo imediato. É verdadeiro quando existe um alienígena em mergulho (`has_diving_alien = True`), a diferença horizontal é muito reduzida ($|dx| \leq 1$) e o alienígena está próximo verticalmente ($dy \leq 3$). Esta variável combate um comportamento excessivamente defensivo que o agente desenvolvia sem ela.

A seleção do *alienígena alvo* segue a seguinte prioridade: se existir um alienígena em mergulho, esse é imediatamente selecionado como alvo; caso contrário, seleciona-se o alienígena ativo mais próximo horizontalmente do jogador. Esta heurística foca o agente na ameaça mais imediata.

O estado é representado como uma tupla Python:

```
1 current_state = (has_diving_alien, aligned, target_dx, target_dy,
2                  can_shoot, danger)
```

Listing 1: Construção do estado abstraído

As variáveis `can_shoot` e `danger` são obtidas da seguinte forma:

```
1 can_shoot = (
2     {"action": "shoot"} in self.current_state.get("valid_actions",
3     [])
4 )
5 danger = (
6     has_diving_alien and
7     abs(target_dx) <= 1 and
8     target_dy <= 3
9 )
```

Listing 2: Obtenção das variáveis `can_shoot` e `danger`

O número total de estados possíveis é:

$$2 \times 2 \times 21 \times 11 \times 2 \times 2 = 3\,696 \quad (1)$$

onde os fatores correspondem, respetivamente, a `has_diving_alien`, `aligned`, `target_dx`, `target_dy`, `can_shoot` e `danger`. O espaço de 3 696 estados é suficientemente compacto para Q-Learning tabular.

4 Algoritmo de Aprendizagem: Q-Learning Tabular

4.1 Espaço de Ações

O agente dispõe de quatro ações possíveis:

ID	Ação	Descrição
0	move WEST	Move a nave um passo para a esquerda
1	move EAST	Move a nave um passo para a direita
2	shoot	Dispara um laser na posição atual
3	None	Não faz nada (espera)

Tabela 1: Espaço de ações do agente.

As ações disponíveis em cada estado são filtradas de acordo com a lista `valid_actions` recebida do servidor, garantindo que o agente nunca tenta ações inválidas (e.g., disparar quando o *cooldown* está ativo). A variável `can_shoot` é extraída precisamente desta lista para enriquecer a representação de estado.

4.2 Estrutura da Q-Table

A Q-table é implementada como um dicionário Python cujas chaves são tuplas de estado e cujos valores são arrays NumPy de dimensão 4 (uma entrada por ação). As entradas são inicializadas a zero por omissão:

```

1 def get_q_values(self, state):
2     if state not in self.q_table:
3         self.q_table[state] = np.zeros(len(ACTIONS), dtype=np.
4             float64)
5     return self.q_table[state]
```

Listing 3: Inicialização preguiçosa da Q-table

4.3 Atualização da Q-Table

A atualização segue a equação clássica do Q-Learning:

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') \right] \quad (2)$$

onde α é a taxa de aprendizagem (`learning_rate`), γ é o fator de desconto (`discount_factor`), r é a recompensa imediata, s' é o próximo estado e a' é a ação ótima nesse estado. Em caso de episódio terminado (`game_over`), a componente de valor futuro é anulada:

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha r \quad (3)$$

4.4 Política ε -Greedy

A política de exploração segue a estratégia ε -greedy: com probabilidade ε o agente escolhe uma ação aleatória (entre as válidas), e com probabilidade $1 - \varepsilon$ explora a ação com maior valor estimado na Q-table. O valor de ε decai geometricamente a cada atualização pelo fator `epsilon_decay`:

$$\varepsilon \leftarrow \varepsilon \times \delta_\varepsilon \quad (4)$$

com $\delta_\varepsilon = 0.9995$, garantindo uma transição gradual de exploração para exploração.

Hiperparâmetro	Valor	Descrição
<code>learning_rate</code> (α)	0.1	Taxa de aprendizagem
<code>discount_factor</code> (γ)	0.9	Fator de desconto para recompensas futuras
<code>epsilon</code> (ε_0)	1.0	Taxa de exploração inicial
<code>epsilon_decay</code> (δ_ε)	0.9995	Fator de decaimento de ε
<code>epsilon_min</code>	0.01	Valor mínimo de ε

Tabela 2: Hiperparâmetros do algoritmo Q-Learning.

5 Função de Recompensa

A função de recompensa foi desenhada para guiar o agente em três dimensões comportamentais: penalizar a morte, incentivar o alinhamento com alvos e refletir o progresso de pontuação real.

5.1 Componentes da Recompensa

A recompensa é calculada de forma diferida: em cada passo, o agente avalia a transição $(s_{t-1}, a_{t-1}) \rightarrow s_t$, ou seja, a recompensa atribuída resulta da comparação entre o estado anterior e o estado atual, após a ação anterior ter produzido efeito no ambiente. Esta abordagem é necessária porque só após observar o novo estado é possível quantificar o resultado da ação tomada (e.g., saber se o alinhamento melhorou, se a pontuação aumentou, ou se o agente morreu).

1. **Penalidade de morte:** quando o episódio termina com `game_over`, é aplicada uma penalidade de -25 , fortemente desincentivando comportamentos que conduzam à morte.
2. **Custo de ação:** cada ação tem um custo residual de -0.01 (para movimento e disparo) ou -0.05 (para não fazer nada), penalizando levemente a inação e incentivando o agente a agir proativamente.
3. **Recompensa de alinhamento:** a diferença absoluta em x entre o agente e o alienígena alvo é comparada entre estados consecutivos: se o agente reduziu a distância horizontal, recebe $+0.1$; se a aumentou, recebe -0.1 . Isto orienta o movimento lateral para uma posição de tiro.
4. **Recompensa de pontuação:** o delta de pontuação entre o estado atual e o anterior (Δscore) é adicionado diretamente à recompensa, alinhando o sinal de aprendizagem com o objetivo explícito do jogo.

A recompensa total numa transição é:

$$r = r_{\text{morte}} + r_{\text{ação}} + r_{\text{alinhamento}} + \Delta \text{score} \quad (5)$$

5.2 Motivação para as Variáveis `can_shoot` e `danger`

A introdução da variável `can_shoot` revelou um problema com a função de recompensa original: a penalização elevada por morte combinada com o aumento do tempo de recarga levava o agente a adotar um comportamento excessivamente defensivo. Quando um alienígena se aproximava na diagonal, o agente tendia a desviar-se mesmo quando poderia disparar, pois não dispunha de informação sobre a disponibilidade do disparo no estado.

A variável `danger` resolve este comportamento ao distinguir situações de ameaça real daquelas em que o alienígena ainda está suficientemente distante. Apenas quando `has_diving_alien = True`, $|dx| \leq 1$ e $dy \leq 3$ em simultâneo é que a situação é classificada como perigosa, encorajando o agente a disparar ativamente nos outros casos em vez de recuar preventivamente.

5.3 Condição de Paragem por Desempenho

Durante o treino, o agente monitoriza continuamente a pontuação. Quando esta atinge ou ultrapassa 15 000 pontos, o treino é interrompido automaticamente, o modelo é guardado e o processo termina. Esta condição garante que o agente não continue a treinar além do necessário.

6 Modos de Operação e Persistência

O agente suporta dois modos de operação, seleccionáveis por argumento de linha de comandos:

- **Modo train:** o agente aprende a partir do zero (ou retoma um ficheiro existente), explorando o ambiente e atualizando a Q-table a cada transição. O treino é realizado manualmente, clicando em *Play* no *frontend* para iniciar cada episódio. No final de cada episódio, o modelo é persistido em disco;
- **Modo play:** o modelo guardado é carregado e o parâmetro ε é fixado a 0.0, forçando a política puramente greedy sem qualquer exploração.

A persistência é feita via *pickle* sobre o dicionário `self.q_table`, guardado por omissão em `agent.pkl`:

```
1 def save(self, file_path: str) -> None:
2     with open(file_path, 'wb') as f:
3         pickle.dump(self.q_table, f)
4
5 def load(self, file_path: str) -> None:
6     with open(file_path, 'rb') as f:
7         self.q_table = pickle.load(f)
```

Listing 4: Serialização e deserialização da Q-table

7 Resultados e Avaliação

7.1 Comportamento Observado

Após treino, o agente demonstra um comportamento coerente com os objetivos definidos: move-se lateralmente em direção ao alienígena mais próximo, aguarda o alinhamento horizontal antes de disparar, e reage a alienígenas em mergulho priorizando-os como alvo imediato. Graças à variável `danger`, o agente distingue ameaças reais de aproximações ainda distantes, disparando ativamente quando não está em perigo imediato em vez de recuar preventivamente. A política aprendida é visivelmente superior à do agente aleatório fornecido, que não exhibe qualquer direcionamento de movimento ou disparo.

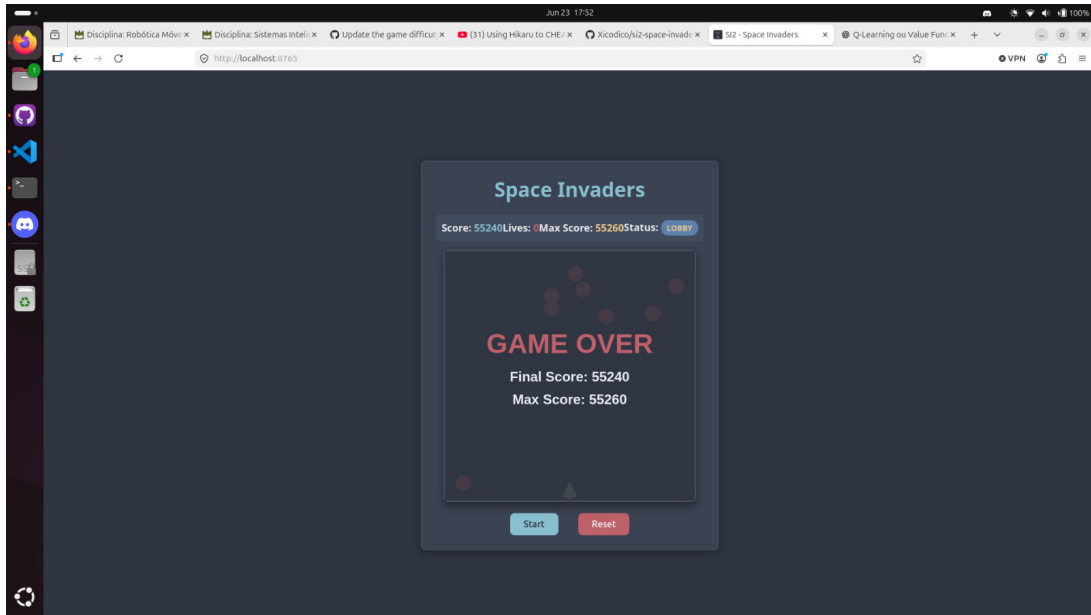


Figura 2: Ecrã de fim de jogo com a pontuação máxima obtida durante o treino (55 260 pontos, alcançada ao longo de 7 episódios / 21 vidas).

7.2 Limitações

A principal limitação da abordagem tabular é a incapacidade de generalizar entre estados similares. Estados não visitados durante o treino têm valores Q inicializados a zero, o que pode levar a comportamentos subótimos em configurações de jogo não encontradas anteriormente.

8 Utilização de Ferramentas de Apoio

Recorreu-se a ferramentas baseadas em modelos de linguagem (LLMs), utilizadas exclusivamente para apoio na identificação e correção de pequenos erros de implementação, bem como na formatação de algumas secções do relatório.